Chapter 12
Parallel Programming

Abstract Parallel programming deals with multi-processing and multi-threading. With the advent of multi-core computers, parallel programming has become essential. In this chapter we discuss the POSIX threading library (pthread). User annotated compiler supported parallelism with OpenMP is described with the help of examples in Section 12.2. The new features of OpenMP version 3.0 (task computing) is presented with the help of examples in Section 12.2.0.3. In addition to multi-threading, parallel computing has been successfully deployed on cluster grids (called the Beowulf class). The most common API used in cluster computing is MPI (message passing interface) which is discussed in this chapter. In addition to these well established parallel programming systems, the rapid rise of many-core and other forms of parallelism has also created new systems which have had less exposure. In particular the Intel Thread Building Block library, and GPGPU computing with NVIDIA CUDA and OpenCL are described.

Contents

12.1 POSIX Thread Library (pthreads) 214
12.2 OpenMP: Open specification for Multi-processing 218
12.3 MPI: Message Passing Interface 228
12.4 Other libraries and tools 230
12.5 Conclusion 240

In this chapter we discuss the various software tools and libraries available for programming parallel and distributed computers. With the ready availability of dual-core, quad-core and many-core machines in the mainstream, utilizing parallel computation paradigms effectively is important. In this Part we will discuss POSIX threading library (pthreads), OpenMP, MPI, Intel TBB, and general purpose computing on graphics card using CUDA and OpenCL.

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_12, © Springer Science+Business Media, LLC 2011

12.1 POSIX Thread Library (pthreads)

Even before we delve deep into the programming model of pthreads, it is useful to see what are the knobs and tuning parameters available to us to manage pthreads. These management methods are independent of the CPU thread scheduling policies or data-cache touching functions.

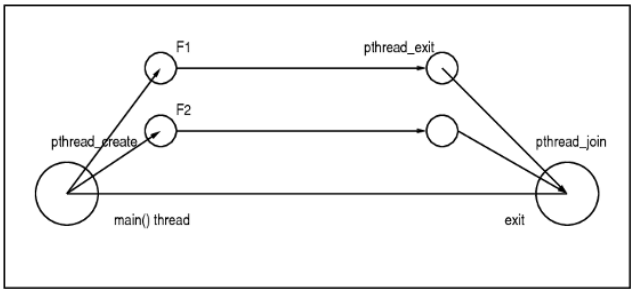


Fig. 12.1 Example of pthread spawning 2 threads for functions F1 and F2.

POSIX pthread library allows management of threads by specifying the following:

1. Keys: threads use keys to maintain private copies of shared data items. A single globally defined key points to different memory location for each thread. We can use keys when making function calls and we want thread specific data to be passed along,
2. Attributes: thread attributes allow us to control scheduling behavior, stack size and initial state,
3. pthread_once mechanism: is like the singleton constructor concept, it ensures that a function or action is performed *exactly once* regardless of how many times (or different threads) attempt to call it. Useful for file or network handling, use this mechanism instead of writing static variables for this purpose,
4. Thread cancellation: allows a thread to self-destruct, for example when searching for a data in a binary-tree, once the data is found, the rest of the threads can be canceled,
5. Thread scheduling: specifies to the Operating System which thread you (as a programmer, not as a user, unless you make the thread scheduling visible to the end-user) would like the CPU to be scheduled. CPU has extensive control on the scheduling behavior of threads.

12.1.1 Understanding pthread programming model

Since pthread is an API library, all of its functionality is exposed using standardized function calls and these calls make up the programming model for pthreads.

The pthread_create() function creates a new thread, with attributes specified by attr, within a process. If `attr` is NULL, the default attributes shall be used. If the attributes specified by `attr` are modified later, the thread's attributes shall not be affected. Upon successful completion, pthread_create() store the ID of the created thread in the location referenced by thread.

```
int pthread_create(pthread_t *restrict thread,
      const pthread_attr_t *restrict attr,
      void *(*start_routine)(void*), void *restrict arg);
```

Attributes objects are provided for threads, mutexes, and condition variables as a mechanism to support probable future standardization and customization. Attributes objects provide clean isolation of the configurable aspects of threads. For example, stack size is an important attribute of a thread, but it cannot be expressed portably.

The pthread_attr_init() function shall initialize a thread attributes object attr with the default value for all of the individual attributes used by a given implementation. The pthread_attr_destroy() function shall destroy a thread attributes object.

```
int pthread_attr_destroy(pthread_attr_t *attr);
int pthread_attr_init(pthread_attr_t *attr);
```

```
#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <pthread.h>
int r1 = 0, r2 = 0;
static void F1(int *pr1) {
  // some compute intensive function
}
static void F2(int *pr2) {
  // some other compute intensive function
}
int main(int argc, char **argv) {
  pthread_t t1, t2;
  if(pthread_create(&t1,NULL,(void *) F1,(void *) &r1) != 0)
    perror("pthread_create"), exit(1);
  if(pthread_create(&t2,NULL,(void *) F2,(void *) &r2) != 0)
    perror("pthread_create"), exit(1);
  if(pthread_join(t1, NULL) != 0)
    perror("pthread_join"),exit(1);
  if (pthread_join(t2, NULL) != 0)
    perror("pthread_join"),exit(1);
  return (EXIT_SUCCESS);
}
```

Listing 12.1 Introduction to pthreads

```
int pthread_once(pthread_once_t *once_control,
       void (*init_routine)(void));
pthread_once_t once_control = PTHREAD_ONCE_INIT;
```

Usually the controlling thread in the CPU will perform setup tasks, but this can be sub-optimal, especially when the setup is compute intensive or involves dealing with blocking I/O.

12.1.2 Pthreads Keys: using thread specific data

As threads are created, run, and destroyed, their working space consists of their stack space, global variables, and memory allocated from the heap. During the course of program execution if you want to associate thread specific data to each thread, so that the same variable in different threads points to different memory, you can use *keys* to achieve this. Consider the example of searching for a value in a tree, every thread function will store its own fragment of the binary tree on which its operating. This can be done using keys:

```
static pthread_key_t tree_key;
int search_init() {
  pthread_key_create( &tree_key, (void*)free_key );
}
```

This function needs the variable for the key as well as a destructor function. Since thread cancellations and exits can leak thread specific data pointed to by keys, implementing this destructor correctly helps prevent memory leaks. Now we can use this key as follows:

```
pthread_setspecific( tree_key, (int*)(binary_tree+offset));
```

We set this threads `tree_key` to `binary_tree`offset+, later on we can use this key to retrieve the binary tree offset as follows:

```
int *tree_offset;
pthread_getspecific( tree_key, (void**)&tree_offset);
```

```
void *pthread_getspecific(pthread_key_t key);
int pthread_setspecific(pthread_key_t key, const void *value);
```

12.1.3 Pthreads Summary

Detailed function description of pthread functions can be found using the `man pthread_create` manpage and then following the related sections. In this section we present the most often used pthread functions, and their description for ready reference.

```
#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <pthread.h>
#define N 100
typedef int matrix_t[N][N];
typedef struct {
  int       id,size,Arow,Bcol;
  matrix_t  *MA, *MB,*MC,dummy;
} arg_t;
matrix_t MA,MB,MC;
void mult(int size,int row,int col,
          matrix_t A, matrix_t B,matrix_t C) {
```

Table 12.1 Pthread Functions

Name	Description
pthread_create	creates thread (pthread_t,attr,function,arg)
pthread_exit	exits thread (status)
pthread_attr_init	initializes attribute (pthread_attr_t*)
pthread_attr_destroy	destroys attribute (pthread_attr_t*)
pthread_join	blocks parent till *T* terminates (pthread_t *T*, void**)
pthread_detach	storage can be claimed when thread terminates (pthread_t),
pthread_self	return the thread id of calling thread (pthread_t)
pthread_equal	compare thread id (pthread_t t1, pthread_t t2)
pthread_control	allows 1 call to function (pthread_once_t *, void* ())
pthread_mutex_init	initializes mutex with attributes (pthread_mutex_t*, pthread_mutexattr_t*)
pthread_mutex_destroy	destroys mutex (pthread_mutex_t*)
pthread_mutexattr_init	initializes mutex attribute (pthread_mutexattr_t*)
pthread_mutexattr_destroy	destroys mutex attribute (pthread_mutexattr_t*)
pthread_mutex_lock	acquire a lock on a mutex (pthread_mutex_t*)
pthread_mutex_trylock	attemp to acquire a lock (pthread_mutex_t*)
pthread_mutex_unlock	unlock the mutex (pthread_mutex_t*)
pthread_cond_init	initializes cond with attributes (pthread_cond_t*, pthread_condattr_t*)
pthread_cond_destroy	destroys cond (pthread_cond_t*)
pthread_condattr_init	initializes cond attribute (pthread_condattr_t*)
pthread_condattr_destroy	destroys cond attribute (pthread_condattr_t*)
pthread_cond_wait	blocks calling thread until condition (pthread_cond_t*, pthread_mutex_t*)
pthread_cond_signal	wakeun up thread waiting for cond (pthread_cond_t*)
pthread_cond_broadcast	waken up all threads waiting on cond (pthread_cond_t*)

```
  int pos;
  C[row][col] = 0;
  for(pos = 0; pos < size; ++pos)
    C[row][col] += A[row][pos] * B[pos][col];
}
void *mult_worker(void *arg) {
  arg_t *p=(arg_t *)arg;
  mult(p->size, p->Arow, p->Bcol, *(p->MA), *(p->MB), *(p->MC));
  free(p);
  return(NULL);
}
int main(int argc, char **argv) {
  int       size, row, col, num_threads, i;
  pthread_t *threads;
  arg_t *p;
  unsigned long thread_stack_size;
  pthread_attr_t *pthread_attr_p, pthread_custom_attr;
  size = N;
  threads = (pthread_t *)malloc(size*size*sizeof(pthread_t));
  for (row = 0; row < size; ++row)
    for (col = 0; col < size; ++col)
```

```
      MA[row][col] = 1, MB[row][col] = row + col + 1;
  num_threads = 0;
  for(row = 0; row < size; row++)
    for (col=0;col<size;col++,num_threads++) {
      p = (arg_t *)malloc(sizeof(arg_t));
      p->id = num_threads;p->size = size;
      p->Arow = row;p->Bcol = col;
      (p->MA) = &MA;(p->MB) = &MB;(p->MC) = &MC;
      pthread_create(&threads[num_threads],&pthread_custom_attr,
                     mult_worker,(void *) p);
  }
  for (i = 0; i < (size*size); i++)
    pthread_join(threads[i], NULL);
  return (EXIT_SUCCESS);
}
```

Listing 12.2 Parallel matrix multiple

12.2 OpenMP: Open specification for Multi-processing

OpenMP has become the defacto standard for shared-memory parallel programming. The last few years has seen OpenMP emerge as the preferred standard, and with the advent of multi-core computing on every desktop, parallel programming with OpenMP is the method of choice for application developers. OpenMP is an Application Program Interface (API) defined by a standardization body comprised of hardware and software vendors. It provides a simple, flexible, yet efficient model for developing portable and scalable parallel applications. GNU GCC version 4.5.0 was used as the C/C++/FORTRAN compiler which supports the latest OpenMP standard (OpenMP ver 3.0). OpenMP comprises of:

1. Compiler directives (written as # pragmas),
2. Runtime library routines: provided by the compiler or hardware provider,
3. Environment variables: controlling aspects of OpenMP,
4. Programming Model: is based on shared-memory, thread based explicit parallelism with a *fork- join* model.

The compiler directives for OpenMP are written as # `pragmas` for the compiler. Consider a small program written in C++ using OpenMP as shown in Listing 12.3.

```
// \file hw_open_mp.cpp
// \author Sandeep Koranne, (C) 2010
// \description Initial program of OpenMP
#include <iostream>           // for program IO
#include <cassert>            // assertions
#include <cstdlib>            // exit
#include <omp.h>              // OpenMP

int main( int argc, char *argv [] ) {
  int num_threads, id;

#pragma omp parallel private( num_threads, id )
  {
    id = omp_get_thread_num();
    std::cout << "Thread id " << id << " says hello!\n";
```

```
    if( id == 0 ) { // master thread
      num_threads = omp_get_num_threads();
      std::cout << "Number of threads = " << num_threads;
    }
  } // join point of all threads

  return ( 0 );
}
```

Listing 12.3 Hello world style program using OpenMP

We can compile and run this program with GCC 4.5.0 as:

```
$g++ -fopenmp hw_open_mp.cpp -o hw_omp
$export OMP_NUM_THREADS=2
$./hw_omp
Thread id 1 says hello!
Thread id 0 says hello!
Number of threads = 2
```

Using the `ldd` program we can check that this binary links to both `libgomp` (which provides the OpenMP run-time library), and `libpthread` (which provides the underlying threading implementation on GNU/Linux, although this is not required by OpenMP).

12.2.0.1 OpenMP directives

OpenMP directives are case sensitive, follow the C/C++ compiler standards for #pragmas. Directives are specified for one structured block by a single line preceding it (if the line becomes too long a line continuation character has to be inserted). Some common OpenMP directives are listed below in Table 12.3.

12.2.0.2 Parallel region construct

defines a block of code which will be executed by multiple threads. It accepts a conditional test clause (to trigger parallel execution), a list of thread-local private variables, shared variables. The number of threads for the block can also be specified, as well as advanced parameters dealing with initialization of variables going into the block (firstprivate) There is an implied barrier at the end of the block.

```
#pragma omp parallel default(shared) private(..)
```

The number of threads can be specified in the parallel construct.

The *data* clause of the parallel directive can be chosen depending on the use of the variable. In OpenMP 3.0 the following data clauses are available:

1. private: variable is localized to each thread,
2. firstprivate: variable is localized and initialized with the value of the variable going into the parallel section,

Table 12.2 GNU libomp runtime functions

Directive	Description
`omp_get_active_level`	Number of active parallel regions
`omp_get_ancestor_thread_num`	Ancestor thread ID
`omp_get_dynamic`	Dynamic teams setting
`omp_get_level`	Number of parallel regions
`omp_get_max_active_levels`	Maximal number of active regions
`omp_get_max_threads`	Maximal number of threads of parallel region
`omp_get_nested`	Nested parallel regions
`omp_get_num_procs`	Number of processors online
`omp_get_num_threads`	Size of the active team
`omp_get_schedule`	Obtain the runtime scheduling method
`omp_get_team_size`	Number of threads in a team
`omp_get_thread_limit`	Maximal number of threads
`omp_get_thread_num`	Current thread ID
`omp_in_parallel`	Whether a parallel region is active
`omp_set_dynamic`	Enable/disable dynamic teams
`omp_set_max_active_levels`	Limits the number of active parallel regions
`omp_set_nested`	Enable/disable nested parallel regions
`omp_set_num_threads`	Set upper team size limit
`omp_set_schedule`	Set the runtime scheduling method
`omp_init_lock`	Initialize simple lock
`omp_set_lock`	Wait for and set simple lock
`omp_test_lock`	Test and set simple lock if available
`omp_unset_lock`	Unset simple lock
`omp_destroy_lock`	Destroy simple lock
`omp_init_nest_lock`	Initialize nested lock
`omp_set_nest_lock`	Wait for and set simple lock
`omp_test_nest_lock`	Test and set nested lock if available
`omp_unset_nest_lock`	Unset nested lock
`omp_destroy_nest_lock`	Destroy nested lock
`omp_get_wtick`	Get timer precision.
`omp_get_wtime`	Elapsed wall clock time.

3. lastprivate: variable is localized, the last iteration copies the value of the variable to the corresponding variable outside the parallel block,
4. shared: all threads in the team share the variable (same memory reference),
5. default: changes the default for all variables in the parallel block,
6. reduction: each thread has a local copy which is combined (*reduced*) using the specified binary operator (only simple binary operator such as +, - are supported),
7. copying: the value of the master thread variable is copied to all threads.

Table 12.3 OpenMP directives

Directive	Description
`parallel`	parallel region construct
`for`	for-loop with implied barrier at end
`sections`	code executed in parallel
`single`	executed by exactly one thread
`task`	finer schedulable entity (ver 3.0)
`master`	synchronization directive
`critical`	code block with critical section
`barrier`	synchronization primitive
`taskwait`	synchronization primitive
`atomic`	memory location updated atomically
`flush`	specific point at which memory is consistent
`ordered`	loop iteration should execute in serial order
`threadprivate`	variables become thread specific

If *nested regions* are enabled (use the library function `omp_get_nested()` to check this), then parallel sections can be nested. The parallel directive can be combined with the worksharing directives shown in the next section.

Work sharing directives are the work-horses of OpenMP. Using these directives the programmer directs the compiler to create parallel threads. The work sharing construct divides the code block execution between the threads on the team. Work sharing directives can be *iterative* (such as **`do`**, **`while`** and **`for`**), or they can be non-iterative SECTIONS, which are listed in the code block. Each section is executed once by a thread (although the same thread may execute multiple sections).

```
#pragma omp parallel shared(sum,A,count) private(i)
{
  #pragma omp for schedule(static) nowait
  for( i=0; i < count; ++i ) {
     sum += A[i];
  }
} /* end of parallel section */
```

As an example of SECTIONS:

```
#pragma omp parallel shared(sumA,A,subB,B,count) private(i)
{
  #pragma omp sections nowait
  {
     #pragma omp section
     for( i=0; i < count; ++i ) sumA += A[i];

     #pragma omp section
     for( i=0; i < count; ++i ) sumB += B[i];
  } /* end of sections */
} /* end of parallel section */
```

For FORTRAN language there is a special WORKSHARE directive which treats all statements in the enclosed structured code block as independent work units.

There also exists a directive **single** which ensures that the enclosed code is executed by only one thread. The work-sharing directives accept arguments which control the scheduling of the threads and the division of the work between the threads on the team. The schedules are:

1. Static: fixed schedule, depends on the size of the work unit in the iteration, number of threads and chunk size,
2. Dynamic: as threads become free they execute on available data.

A comparison of the scheduling methods available in OpenMP was performed. The workload used was the evaluation of the return-count of the *Collatz* function of a number. The Collatz function is defined as a recurrence:

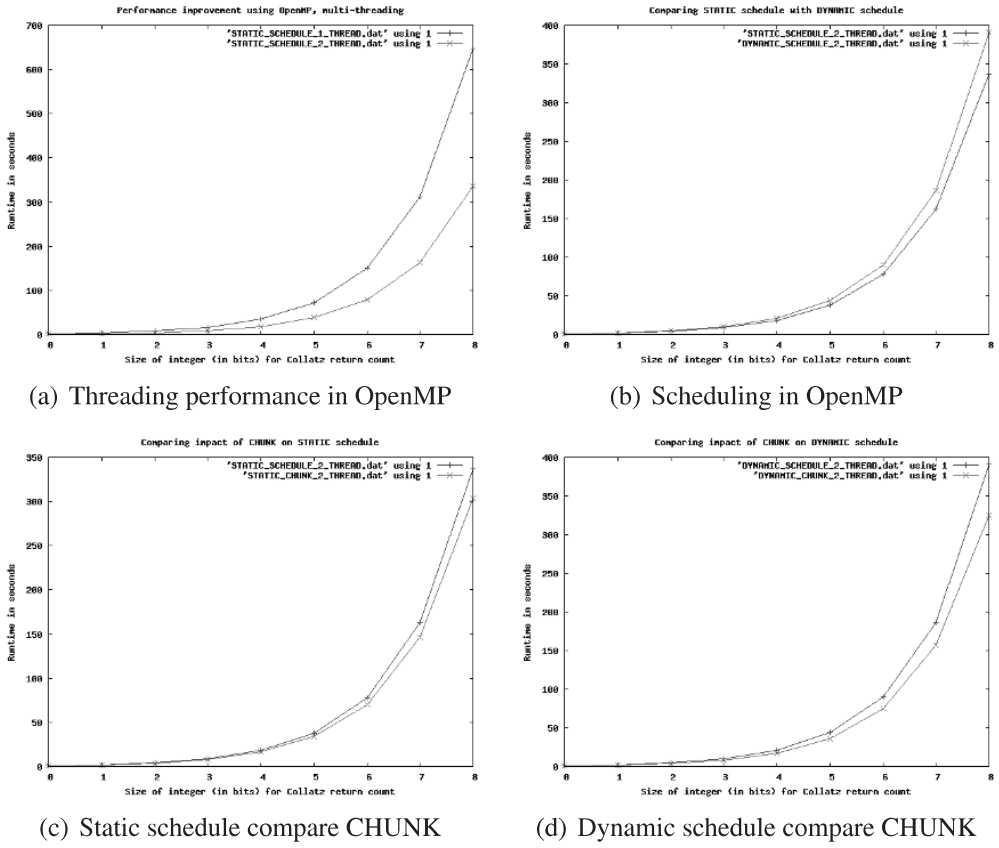
```
collatz(n) = 1      : n == 1
           = n/2    : n is even
           = 3n+1   : n is odd
```

The return-count of the Collatz function is the number of times the function would need to be evaluated given a positive integer as input. Although defined as a recurrence, we implement it as a `while` loop as shown in Listing 12.4.

```
volatile unsigned long CountCollatzReturn( unsigned long N ) {
  static const unsigned int LOOP_COUNT = 1;
  unsigned long count = 0;
  unsigned long storedN = N;
  for( unsigned long j=0; j < LOOP_COUNT; ++j ) {
    N = storedN;
    count=j;
    while( 1 ) {
      count++;
      if( N <= 1 ) break;
      if( N % 2 ) N = 3*N+1;
      else N = N/2;
    }
  }
  return count;
}
```

Listing 12.4 Return count of the Collatz function

We write several OpenMP functions, each differing in the scheduling method. These functions are listed in Listing 12.5. The performance comparison is shown in Figure 12.2. In Figure 12.2(a), the performance benefit of adding another processor to the thread pool is shown. Since the Collatz function is compute bound, we get linear speedup. As stated above, the scheduling of threads can be left as static (using the static schedule in OpenMP). The Collatz function has the property that the return count of any integer is not completely dependent on the size of the integer (e.g., return count for all even numbers is simply $\lg(n)$), thus the static schedule performs adequately. But if the workload was dependent on the size of the integer, then a static schedule would exhibit the *tail syndrome*, where the initial values would be computed quickly, but as the length of the integers increases, the time would increase (e.g., this manifests itself in number theoretic functions checking whether a given number is prime or not). The static schedule divides the iteration count of the



(a) Threading performance in OpenMP

(b) Scheduling in OpenMP

(c) Static schedule compare CHUNK

(d) Dynamic schedule compare CHUNK

Fig. 12.2 Comparing performance in OpenMP using various scheduling and data chunking strategies.

`for` loop by the number of threads (as specified in the parallel section) or based on the `OMP_NUM_THREADS` environment value.

In *dynamic* scheduling, as and when threads get free they take new work from the pool of not yet finished loop iterations. At the expense of managing this dynamic queue, the tail syndrome can be mitigated. The runtime of the function using static schedule is compared with dynamic schedule as shown in Figure 12.2(b). As we discussed above, in this example the static schedule is reasonably *fair*, thus the dynamic schedule's queue management overhead is visible.

By default OpenMP processes a single loop iteration per-thread. If the workload computation is small (as in this case), it is more efficient to process larger *chunks* per thread. This can be specified using the `chunk` construct in the OpenMP pragma. We compare the addition of chunks to static schedule in Figure 12.2(c), and to dynamic schedules in Figure 12.2(d).

```
// experiment to try different schedule
int dynamic_chunk( unsigned long scount ) {
  std::cout << __PRETTY_FUNCTION__ << std::endl;
  unsigned long i;
  unsigned long ans=0;
  int chunk = 100;
```

```
  #pragma omp parallel private(i) shared( scount,chunk,ans )
  {
    #pragma omp for schedule( dynamic,chunk ) nowait
    for( i=0; i < scount; ++i ) {
      unsigned long retCount = CountCollatzReturn( i );
      if( retCount > 400 ) ans++;
    } /* end of parallel section */
  } // join point of all threads
  std::cout << "Ans = " << ans << std::endl;
  return 0;
}

int dynamic_schedule( unsigned long scount ) {
  std::cout << __PRETTY_FUNCTION__ << std::endl;
  unsigned long i;
  unsigned long ans=0;
  #pragma omp parallel private(i) shared( scount,ans )
  {
    #pragma omp for schedule( dynamic ) nowait
    for( i=0; i < scount; ++i ) {
      unsigned long retCount = CountCollatzReturn( i );
      if( retCount > 400 ) ans++;
    } /* end of parallel section */
  } // join point of all threads
  std::cout << "Ans = " << ans << std::endl;
  return 0;
}

int static_chunk( unsigned long scount ) {
  std::cout << __PRETTY_FUNCTION__ << std::endl;
  unsigned long i;
  unsigned long ans=0;
  int chunk = 100;
  #pragma omp parallel private(i) shared( scount,chunk,ans )
  {
    #pragma omp for schedule( static, chunk) nowait
    for( i=0; i < scount; ++i ) {
      unsigned long retCount = CountCollatzReturn( i );
      if( retCount > 400 ) ans++;
    } /* end of parallel section */
  } // join point of all threads
  std::cout << "Ans = " << ans << std::endl;
  return 0;
}


int static_schedule( unsigned long scount ) {
  std::cout << __PRETTY_FUNCTION__ << std::endl;
  unsigned long i;
  unsigned long ans=0;
  #pragma omp parallel private(i) shared( scount,ans )
  {
    #pragma omp for schedule( static ) nowait
    for( i=0; i < scount; ++i ) {
      unsigned long retCount = CountCollatzReturn( i );
      if( retCount > 400 ) ans++;
    } /* end of parallel section */
  } // join point of all threads
  std::cout << "Ans = " << ans << std::endl;
  return 0;
}

int static_wait( unsigned long scount ) {
  std::cout << __PRETTY_FUNCTION__ << std::endl;
  unsigned long i;
  unsigned long ans=0;
  #pragma omp parallel private(i) shared( scount,ans )
```

```
  {
    #pragma omp for schedule( static )
    for( i=0; i < scount; ++i ) {
      unsigned long retCount = CountCollatzReturn( i );
      if( retCount > 400 ) ans++;
    } /* end of parallel section */
  } // join point of all threads
  std::cout << "Ans = " << ans << std::endl;
  return 0;
}

int dynamic_wait( unsigned long scount ) {
  std::cout << __PRETTY_FUNCTION__ << std::endl;
  unsigned long i;
  unsigned long ans=0;
  #pragma omp parallel private(i) shared( scount,ans )
  {
    #pragma omp for schedule( dynamic )
    for( i=0; i < scount; ++i ) {
      unsigned long retCount = CountCollatzReturn( i );
      if( retCount > 400 ) ans++;
    } /* end of parallel section */
  } // join point of all threads
  std::cout << "Ans = " << ans << std::endl;
  return 0;
}


int main( int argc, char * argv[] ) {
  int shift_count = atoi( argv[1] );
  unsigned long scount = 1UL << shift_count;
  int which_algo = atoi( argv[2] );
  switch( which_algo ) {
  case 0: { return static_schedule( scount ); }
  case 1: { return dynamic_schedule( scount); }
  case 2: { return static_chunk( scount ); }
  case 3: { return dynamic_chunk( scount); }
  case 4: { return static_wait( scount); }
  case 5: { return dynamic_wait( scount); }
  default:{ return static_schedule( scount ); }
  }
  return 0;
}
```

Listing 12.5 Experimenting with OpenMP scheduling

The *shared* and *private* lists of data in the parallel sections control the data visibility of variables to the threads. A variable present in the 'private' list implies that all threads receive a new object of the same type (value of this variable is uninitialized, unless *copying* or *firstprivate* is specified). Shared variables, on the other hand, refer to the same location in memory, and all threads can access the same variable. A variable may also be passed as a *reduction* variable with an associated operator. OpenMP currently limits the operator to be arithmetic operations such as addition, subtraction, and bitwise Boolean. Although, sufficient for basic purposes such as scans and prefix-sums, a generic binary reduction operator as a C++ template type can easily be written. An example is shown in Listing 12.6, which computes the minimum value of a given vector.

```
// \file min_omp.cpp
// \author Sandeep Koranne, (C) 2010
// \description Calculate minimum value of vector
#include <omp.h>
```

```
#include <iostream>
#include <cstdlib>
#include <limits>

template <typename T>
T MyOp( T a, T b ) { return std::min( a, b ); }

// Although OpenMP does not support reduction on
// arbitrary functions, it is simple to code it.

int main( int argc, char *argv [] ) {
  if( argc != 2 ) {
    std::cerr << "Usage: <N>\n";
    exit(1);
  }
  int i;
  int N = atoi( argv[1] ); // number of elements
  int *data = new int[N];
  for( i=0; i < N; ++i ) data[i] = rand() ^ 177317;
  int chunk = N/10; // chunk size
  int result = std::numeric_limits<int>::max();

  #pragma omp parallel private(i) shared(N,chunk,result)
  {
    int local_min = std::numeric_limits<int>::max();
    #pragma omp for schedule(static,chunk) nowait
       for(i=0; i < N; ++i) local_min = MyOp( local_min, data[i] );

    #pragma omp critical
    {
      result = MyOp( result, local_min );
    }
  }
  std::cout << "Minimum value = " << result << std::endl;
  delete [] data;
  return (0);
}
```

Listing 12.6 OpenMP reduction style min operator

The use of a local variable to accumulate partial results per thread which are then *merged* with the final result under a critical section is a powerful idiom. This can reduce locking on the shared variable, and thus improve performance.

12.2.0.3 Task parallelism in OpenMP ver 3.0

New in OpenMP 3.0 is the concept of task parallelism. This was added in response to the increasing demand of application writers who had to deal with irregular parallelism. The *Task* construct defines an explicit task (which is then executed by some thread), which shares data according to the data sharing attributes (as discussed above, see *shared*, and *private*).

The *fork* and *join* concept of OpenMP is exemplified in the design of the TASK construct. See Figure 12.3; an example of using task parallelism in OpenMP is shown in Listing 12.7.

```
// \file task_example.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of task based parallelism
#include <iostream>          // for program IO
```

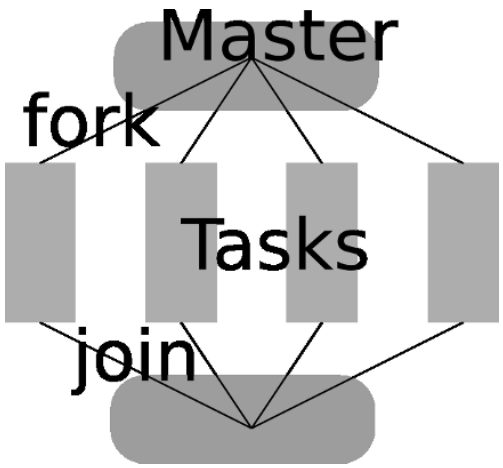


Fig. 12.3 Task parallelism in OpenMP 3.0

```
#include <vector>              // data storage
#include <algorithm>           // for copy
#include <iterator>            // ostream iterator
#include <omp.h>               // OpenMP

static void AnalyzeData( std::vector<int>& D,
                         int index,
                         const unsigned int N,
                         const unsigned int chunk ) {
  // each chunk = [N/chunk]*index, [N/chunk]*(index+1)
  unsigned int start_index = (N/chunk) * index;
  unsigned int stop_index = start_index + (N/chunk);
  for( unsigned int i=start_index; i < stop_index; ++i )
    D[i] = index;
}

void ExecuteFunction(void) {
  static const unsigned int N = 48;
  std::vector<int> Data( N );
  #pragma omp parallel
  {
    #pragma omp single nowait
    {
      for( int i=0; i < N; ++i ) Data[i]=i;
    }
    // create 16 tasks, each responsible for doing part of the work
    for( int i=0; i < 16; ++i ) {
    #pragma omp task if( i > 10 )
      {
        AnalyzeData( Data, i, N, 16 );
        #ifdef _OPENMP
        std::cout<<"\nComputing with "<<i<<"\t"<<omp_get_thread_num();
        #endif
      }
    }
    #pragma omp taskwait
  }
  std::copy( Data.begin(), Data.end(),
             std::ostream_iterator<int>( std::cout, " " ) );
}

int main( int argc, char *argv [] ) {
  ExecuteFunction();
```

```
    std::cout << std::endl;
50  return (0);
}
```

Listing 12.7 Example of TASK parallelism in OpenMP

The TASK construct has an optional scalar expression which controls whether the task should be executed on a new thread or executed on the existing thread. On entry upon the parallel sections, already threads have been created, and if the scalar expression returns false, the task code block is executed in-situ.

OpenMP also supports a *Critical* construct which should enclose a block of code which must be executed only by one thread at any given time. Example can be a function to write messages to log file. Task synchronization is achieved using the *Barrier* directive which requires threads to wait at the barrier until all other threads also reach that barrier, at which time all the threads resume execution.

12.3 MPI: Message Passing Interface

Before multicore computing became commonplace in the last 2-3 years, large scale high performance computing projects relied on *cluster computing* or *distributed* computing. A cluster of 100 nodes proved more beneficial (flops per dollar cost) to the end user, as compared to an expensive shared memory multi-CPU machine with 100 CPUs. As the number of vendors deploying clusters increased, a need for standardized APIs for messaging arose. The MPI (message passing interface) API was deigned in response to this need and it comprises of a number of functions as listed below. The functions are divided into categories of:

1. Environment functions: to initialize the MPI execution environment, determine the size of the process group, and compute the rank,
2. C/C++ Library API: includes the point-to-point communication, `MPI_Send` and `MPI_Recv`.
3. Run time library,
4. Configuration file.

12.3.0.4 MPI Environment functions

The environment functions include `MPI_Init()`, `MPI_Comm_size()`, `MPI_Abort()`, and `MPI_Finalize()`.

12.3.0.5 C/C++ Library API

The MPI library contains functions for blocking and non-blocking message passing. As a communication library, MPI ensures (a) ordering of messages and (b) fairness

of messages. The MPI functions of `MPI_Send()`, and `MPI_Recv()` can be used. Messages to the same host can be differentiated using *tags* included with the message, as:

```
MPI_Send( &buf, count, datatype, dest, tag, communication);
```

Consider an example:

```
  MPI_Init(&argc,&argv);
  MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);

  if( rank == 0 ) {
    rc = MPI_Send(&outmsg, 1, MPI_CHAR, dest,
                  tag, MPI_COMM_WORLD);
    rc = MPI_Recv(&inmsg, 1, MPI_CHAR, source,
                  tag, MPI_COMM_WORLD, &Stat);
  }
```

12.3.0.6 Run-time library

The compiler options to compile the C/C++ program and link it with the appropriate MPI library can be done automatically using the `mpicc` command.

12.3.0.7 Configuration file

The `mpd.hosts` file lists the hosts on which MPI programs can run. The port range and password for authentication is listed in `.mpd.conf`. Thereafter, the `mpd` program can be launched, either using `mpdboot` command or direct invocation of the `mpd` program. The servers can be linked together into the same MPI ring. Consider the program as shown in Listing 12.8.

```
/*
 * \file mpi_hw.c
 * \author Sandeep Koranne (C) 2010
 * \description MPI hello_world program
 */
#include <mpi.h>              /* for MPI */
#include <stdio.h>            /* for program IO */
#include <stdlib.h>           /* for exit */

int main( int argc, char *argv[] ) {
  int number_tasks, rank, rc;
  rc = MPI_Init( &argc, &argv );
  if( rc != MPI_SUCCESS ) {
    perror("MPI init failed");
    MPI_Abort( MPI_COMM_WORLD, rc );
    exit( 1 );
  }

  MPI_Comm_size( MPI_COMM_WORLD, &number_tasks );
  MPI_Comm_rank( MPI_COMM_WORLD, &rank );
  printf("\n Num tasks = %d, rank = %d",
   number_tasks, rank );
  MPI_Finalize();
```

```
    return (0);
25  }
```

Listing 12.8 Example program using MPI

Running this program on our MPI cluster prints the number of tasks and the rank of the process. It should be noted that MPI itself is only a specification, and an *implementation* is provided by a specific vendor (hardware or software), or a generic version such as MPICH can be used on the existing cluster. Since MPI was designed for primarily message passing, it has the concept of a *communicator group* which is an object that specifies the collection of MPI processes which may communicate with each other. Moreover, each process has an unique integer identifier denoted *rank* which can be used for communication.

12.3.1 Using Boost.MPI

In addition to using the default MPI API, we can also write MPI programs in C++ using Boost. The Boost.MPI is a library for message passing built on top of MPI. Boost.MPI is a C++ interface which provides an alternative style of programming and development style including (a) support for user-defined data types, (b) C++ STL, (c) arbitrary function objects, and (d) template functions. The functions of MPI.Boost can be classified the same as MPI:

1. Communicators: creation, cloning, and splitting of MPI communicators, manipulation of process groups,
2. Point-to-point communication: communication of primitive and user-defined data types with send and receive operation with blocking and non-blocking interfaces,
3. Collective communications: including *reduce* and *gather*,
4. MPI Datatypes.

Boost.MPI can be directly used through C++ or using Python interface.

12.4 Other libraries and tools

In this section we discuss some of the new parallel programming libraries and software.

12.4.1 Thread Building Blocks

Thread Building Blocks is an C++ library by Intel designed to improve the efficacy of expressing parallelism using C++. Intel TBB is written using modern C++ tech-

niques such as *templates*, and object oriented programming. TBB is written to allow the expression of task based, as well the *plain-old* thread based parallelism. It supports parallel design patterns and promotes scalable data parallel programming. As we discussed above in the case of OpenMP (see Section 12.2), TBB also specifies tasks, as opposed to threads, TBB manages the scheduling, which in the general case results in better performance; however, TBB has excellent support for C++, as shown in Listing 12.9.

```
// \file tbb_example.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of using Thread Building Block library

#include <iostream>           // for program IO
#include <string>             // for std::string
#include <cassert>            // assertion checking
#include <cstdlib>            // exit
#include <algorithm>          //
#include "tbb/parallel_for.h"
#include "tbb/blocked_range.h"

class NoWorkDone {
public:
  void operator()( const tbb::blocked_range<size_t>& R ) const {
    //std::cout << std::endl << "Range = " << R.begin() << "\t" << R.end();
  }
};


class TakeuchiBenchmark {
  static unsigned int Takeuchi( int x, int y, int z) {
    if( y >= x ) return z;
    // extra looping constructs, to add computation time
    #if 0
    for( unsigned int i=0; i < z; ++i )
      for( unsigned int j=0; j < x; ++j )
        x = ( x & i) | x, y = y & j | y;
    #endif
    return Takeuchi( Takeuchi( x-1, y, z ),
                     Takeuchi( y-1, z, x ),
                     Takeuchi( z-1, x, y ));
  }

public:
  void operator()( const tbb::blocked_range<size_t>& R ) const {
    //std::cout << std::endl << "Range = " << R.begin() << "\t" << R.end();
    unsigned long int sum = 0;
    for( size_t i = R.begin(); i < R.end(); ++i ) {
      sum += Takeuchi(i, i+1, i+2 );
    }
    //std::cout << "\n Sum = " << sum << std::endl;
  }
};


int main(int argc, char *argv[]) {
  unsigned int N = 16;
  if( argc == 2 ) N = atoi( argv[1] );
  std::cout << "Calculate schedule.\n";
  tbb::parallel_for( tbb::blocked_range<size_t>( 0, N ),NoWorkDone() );

  std::cout << "Takeuchi benchmark...\n";
  tbb::parallel_for( tbb::blocked_range<size_t>( 0, N ),
                     TakeuchiBenchmark() );
  return ( EXIT_SUCCESS );
```

```
}
```

Listing 12.9 Example of using Intel TBB

As shown in Listing 12.9, the `operator()` function is defined for the class; the TBB library splits the workload using `tbb::blocked_range<size>`. The partitioning scheme can be changed by passing a custom partitioner to the `parallel_for` function. The `parallel_for` function introduces parallelism in the code. TBB also supports *nested parallelism*. A nice use of C++ templates is the *lambda* expression to encapsulate a code block without creating a dummy class with the `operator()`. This features uses recent features of the C++ 00x standard, but is supported in GCC 4.5.0. An example is shown below in Listing 12.10.

```
int main(int argc, char *argv[]) {
  unsigned int N = 16;
  if( argc == 2 ) N = atoi( argv[1] );

  parallel_for( blocked_range<size_t>( 0, N ),
  [=]( const blocked_range<size_t>& R ) {
       unsigned long int sum = 0;
       for( size_t i = R.begin(); i < R.end(); ++i )
         sum += TakeuchiBenchmark::Takeuchi(i, i+1, i+2 );
  }
  );
  return ( EXIT_SUCCESS );
}
```

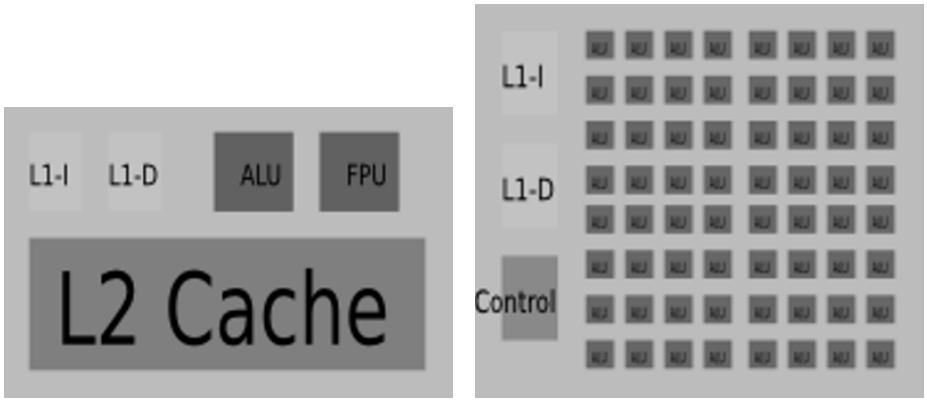
Listing 12.10 Lambda expressions in TBB

As with OpenMP, TBB also defines *reductions* to be performed in parallel. Complete C++ support for binary operator is available, and using this, arbitrary complex binary reductions can be performed in parallel. Another key facility provided by TBB is a memory allocator optimized for multi-threaded allocations.

12.4.2 CUDA : C Unified Device Architecture

A recent entrant to the parallel computing stage is the GPGPU (general purpose computing on graphics processing unit) concept. Graphics chips (such as those by NVIDIA and AMD) have enormous compute power in the form of streaming processors designed for high-performance graphics. As compared to the CPU architecture (shown in Figure 12.4(a)) the GPUs have most of the silicon area dedicated to ALUs (as shown in Figure 12.4(b)). The branch control logic and memory latency hiding caches are smaller. This presents both an opportunity and a challenge, as traditional C/C++ programs are written for execution on a CPU and thus cannot directly take advantage of the massive data-parallelism which is present in the GPU.

Thus, programming GPUs is accomplished by special compilers and tools, although recent progress in auto-parallelizing (at least FORTRAN code) has been made. One such programming system for GPUs is CUDA by NVIDIA. CUDA is an acronym for C Unified Device Architecture. A more standardized (able to execute



(a) CPU Organization (b) GPU Organization

Fig. 12.4 Difference between CPU and GPU organization

on NVIDIA as well as AMD chips) method is to use OpenCL. In this section we describe the use of CUDA and OpenCL to program GPUs.

Our test hardware for the CUDA setup is a GeForce GT 240 graphics card plugged into a PCI-Express slot on an Opteron x86-64 host running GNU/Linux (Fedora Core 12). NVIDIA CUDA ships with a `deviceQuery` utility which can report the various properties of the underlying hardware, this is required when writing programs which are expected to run on generic CUDA hardware. Running this program on our CUDA setup we get:

```
./bin/linux/release/deviceQuery Starting...

 CUDA Device Query (Runtime API) version (CUDART static linking)

There is 1 device supporting CUDA

Device 0: "GeForce GT 240"
  CUDA Driver Version:                           3.10
  CUDA Runtime Version:                          3.10
  CUDA Capability Major revision number:         1
  CUDA Capability Minor revision number:         2
  Total amount of global memory:                 1073020928 bytes
  Number of multiprocessors:                     12
  Number of cores:                               96
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       16384 bytes
  Total number of registers available per block: 16384
  Warp size:                                     32
  Maximum number of threads per block:           512
  Maximum sizes of each dimension of a block:    512 x 512 x 64
  Maximum sizes of each dimension of a grid:     65535 x 65535 x 1
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             256 bytes
  Clock rate:                                    1.34 GHz
  Concurrent copy and execution:                 Yes
```

```
  Run time limit on kernels:                     Yes
  Integrated:                                    No
  Support host page-locked memory mapping:       Yes
  Compute mode:                                  Default
  Concurrent kernel execution:                   No
  Device has ECC support enabled:                No

deviceQuery, CUDA Driver = CUDART, CUDA Driver Version = 3.10,
CUDA Runtime Version = 3.10, NumDevs = 1, Device = GeForce GT 240

PASSED
```

12.4.3 SIMT in CUDA

The CUDA stream multiprocessors create, manage, schedule and execute threads in groups called *warps*. Threads belonging to a warp have the same instruction stream (program code) and start off from the same location. Each thread has its own program counter (PC), register state and thus can branch independently of each other; however, as long the threads execute the same instruction path full efficiency is realized, but as soon as a thread diverges, the warp serializes the execution of thread until the threads synchronize to the same point. This behavior, where a group of threads execute the same instruction, but may diverge out of sequence is termed SIMT (single instruction multiple thread). As long as the programmer can optimize that code path divergence is minimized, efficiency can be optimized, but unlike SIMD, branches can be taken by the threaded code. The ability of the GPU architecture to launch threads and schedule them in hardware makes the runtime penalty of this thread management small compared to execution time of the data-parallel portions of the kernel.

12.4.4 Compute Kernels in CUDA

The key concept in CUDA is the idea of *compute kernels*. Compute kernels are C functions which are executed in parallel by N different CUDA threads. Consider a compute kernel as shown below:

```
__global__ void AddVector( float* A, float* B, float* SUM ) {
  int i = threadIdx.x;
  SUM[i] = A[i] + B[i];
}

int main() {
  AddVector<<<1,N>>>( A, B, SUM );
  // Parallel invocation of N threads
  return (0);
}
```

In the above listing N threads execute the `AddVector` compute kernel in parallel. The `__global__` declaration specifier denotes the function as a CUDA kernel function. Inside the kernel the variable `threadIdx` and `blockIdx` are defined by the CUDA system. These are actually three dimensional vector. Calling the kernel with different blocks is done as:

```
{
   AddVector<<< numBlocks, numThreadsPerBlock >>>(A,B,SUM);
}
```

There is a system imposed limit on the number of threads per block, due in part to the CUDA constraints on limited memory resources on the core. Moreover, thread blocks are required to execute independently, as well as in any order. Threads within a block can cooperate by sharing data through *shared memory* and using synchronization functions.

12.4.5 Compiling CUDA code with NVCC

NVCC is the CUDA compiler driver which compiles C code to a form which can be executed on the GPU. The compiler outputs PTX (assembly) and/or binary instructions from C code containing a mixture of host code and GPU code (kernels).

CUDA functions to allocate memory on the host and device allow the programmer to transfer data for computation on the GPU. Texture memory on the GPU can also be used as a lookup memory for constant data. Consider the following example:

```
// \file cuda_memory.cu
// \author Sandeep Koranne, (C) 2010
// \description CUDA example file

#include <cuda.h>
#include <cuda_runtime_api.h>
#include <iostream>

__global__ void DepictBlockDim( float* SUM, int N ) {
  int i = blockDim.x * blockIdx.x + threadIdx.x;
  if( i < N ) SUM[i] = blockDim.x;
}

__global__ void DepictBlockId( float* SUM, int N ) {
  int i = blockDim.x * blockIdx.x + threadIdx.x;
  if( i < N ) SUM[i] = blockIdx.x;
}

__global__ void DepictIteration( float* SUM, int N ) {
  int i = blockDim.x * blockIdx.x + threadIdx.x;
  if( i < N ) SUM[i] = i;
}

__global__ void AddVector( float* A, float* B, float* SUM, int N ) {
  int i = threadIdx.x;
  SUM[i] = A[i] + B[i];
}

int main() {
  cuInit(0);
```

```
  int deviceCount = 0;
  cudaGetDeviceCount( &deviceCount );
  if( deviceCount == 0 ) {
    std::cerr << "Unable to find any CUDA devices..\n";
    return (1);
  }
  const unsigned int N = 16;
  const size_t size = N*sizeof(float);
  std::cout << deviceCount << " CUDA device(s) found.\n";
  float* A = (float*) malloc( size );
  float* B = (float*) malloc( size );
  float* SUM = (float*) malloc( size );
  for( unsigned int i=0; i < N; ++i ) {
    A[i] = B[i] = 1.0 * i;
    SUM[i] = 0.0;
  }
  float *d_A = NULL, *d_B = NULL, *d_SUM = NULL;
  cudaMalloc( (void**) &d_A, size ); // device allocate
  cudaMalloc( (void**) &d_B, size ); // device allocate
  cudaMalloc( (void**) &d_SUM, size ); // device allocate

  cudaMemcpy( d_A, A, size, cudaMemcpyHostToDevice );
  cudaMemcpy( d_B, B, size, cudaMemcpyHostToDevice );

  //AddVector<<<1,N>>>( d_A, d_B, d_SUM ); // SUM = A + B
  int numThreadsPerBlock = 256;
  int blocksPerGrid = ( N + numThreadsPerBlock -1 )/numThreadsPerBlock;
  std::cout << "Blocks per grid = " << blocksPerGrid << "\n";
  //DepictBlockDim<<<8,8>>> (d_SUM,N);
  //DepictBlockId<<<8,8>>> (d_SUM,N);
  //DepictIteration<<<8,8>>>( d_SUM, N );
  AddVector<<<blocksPerGrid,numThreadsPerBlock>>>( d_A, d_B, d_SUM, N );
  cudaMemcpy( SUM, d_SUM, size, cudaMemcpyDeviceToHost );
  cudaFree( d_A );
  cudaFree( d_B );
  cudaFree( d_SUM );
  for( unsigned int i=0; i < N; ++i ) {
    std::cout << SUM[i] << " ";
  }
  std::cout << std::endl;
  return (0);
}
```

Listing 12.11 Kernel dimensions in CUDA

We compile this CUDA file as:

```
nvcc -I /usr/local/cuda/include/ cuda_memory.cu  -lcuda
```

Listing 12.11 has many different compute kernels. Except for the `AddVector` kernel, the purpose is to demonstrate the effect of kernel *dimensions*, parameters to the CUDA runtime listed within the `<<<B,N>>>` angled brackets. The first argument is the `blocksPerGrid`, and the second argument is `threadsPerBlock`. The kernel implicit variable `blockDim` is defined as the *dimension* of the block, while individual values of the block id are available in the implicit variable `blockIdx`. We selectively run each of these compute kernels, the results are shown below:

Kernel is `DepictBlockDim`

```
$ ./a.out
1 CUDA device(s) found.
Blocks per grid = 2
```

```
8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8
```

The block dimension is indeed 8, as that was passed as the argument to the CUDA runtime.

Kernel is `DepictBlockId`

```
$ ./a.out
1 CUDA device(s) found.
Blocks per grid = 2
0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1
```

Since there are a total of 16 values, and 8 blocks per grid, there will be a total of 2 blocks, of id 0 and id 1. This is what we see from the above output.

Kernel is `DepictIteration`

```
$ ./a.out
1 CUDA device(s) found.
Blocks per grid = 2
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
```

In this kernel we compute the iteration index for each member of the array. Typically, this formulation is used in most kernels, the id of the variable is computed as a function of blockDim, blockId, and threadId. If a multi-dimensioned thread block is used, conventional row-column matrix index conversion formulas can be used to compute the index.

Kernel is `AddVector`

```
$ ./a.out
1 CUDA device(s) found.
Blocks per grid = 2
0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30
```

This kernel performs the actual addition. Since both vectors A and B are sequentially filled, we expect the SUM array to be comprising of sequential even numbers. For high-efficiency, number of threads per block is set to 256.

12.4.5.1 PyCUDA

While CUDA API and the `nvcc` compiler provide complete access to the GPU hardware, programming in CUDA is low-level and time consuming. For numerical applications which involve operations on array, an alternative is to use a Python software library called PyCUDA which is a higher level alternative.

Consider the following example in PyCUDA as shown in Listing 12.12.

```
# \file pycuda_props.py
# \author Sandeep Koranne (C) 2010
# \description Example file using PyCUDA for GP GPU Computing
import pycuda.driver as drv
import pycuda.autoinit
import numpy
```

```
import sys

from pycuda.compiler import SourceModule

mod = SourceModule("""
  __global__ void VectorAdd( float* A, float* B, float* SUM ) {
    const int i = threadIdx.x;
    SUM[i] = A[i] + B[i];
  }

  __global__ void DepictBlockId( float *SUM ) {
     const int i = blockDim.x * blockIdx.x + threadIdx.x;
     SUM[i] = i;
   }
""")

VectorAdd = mod.get_function("VectorAdd")

numCudaDevices = drv.Device.count()

if numCudaDevices == 0:
    print "No CUDA devices detected..\n"
    sys.exit(1)
else:
    print "%d CUDA devices detected." % numCudaDevices
    dev = drv.Device(0)
    print "Compute Capability %d.%d" %dev.compute_capability()
    print "Device has %s MB memory" % (dev.total_memory()//(1024*1024))

# we have the CUDA device
N = 16
a = numpy.random.randn(N).astype(numpy.float32)
b = numpy.random.randn(N).astype(numpy.float32)
sum = numpy.zeros_like( a )
#VectorAdd( drv.In( a ), drv.In( b ), drv.Out( sum ),block=(N,2,1))
DepictBlockId = mod.get_function("DepictBlockId")
DepictBlockId( drv.Out( sum ), block=(N,2,1) )

print sum

sys.exit(0)
```

Listing 12.12 Example of PyCUDA

Running the code in Listing 12.12 we get:

```
$ python pycuda_props.py
1 CUDA devices detected.
Compute Capability 1.2
Device has 1023 MB memory
[   0.    1.    2.    3.    4.    5.    6.    7.    8
...
]
```

Using GPUarrays, the task of running simple compute kernels can be simplified even further as shown in Listing 12.13.

```
# \file gpuarray.py
# \author Sandeep Koranne (C) 2010
# \description Example of using GPUarray in PyCUDA with CUDA
import pycuda.gpuarray as gpuarray
import pycuda.driver as drv
import pycuda.autoinit
```

```
import numpy
import sys

numCudaDevices = drv.Device.count()

if numCudaDevices == 0:
    print "No CUDA devices detected..\n"
    sys.exit(1)

N=4
h_a = numpy.arange(N).astype(numpy.float32)
d_a = gpuarray.to_gpu( h_a )
a_2x = (2*d_a).get()
print "Host Array = "
print h_a
print "After computing"
print a_2x


sys.exit(0)
```

Listing 12.13 Example of GPUarrays in PyCUDA

Running the code in Listing 12.13, we get:

```
$python gpuarray.py
Host Array =
[ 0.  1.  2.  3.]
After computing
[ 0.  2.  4.  6.]
```

Comparing the code in Listing 12.13 to the C code compiled with `nvcc`, one can immediately see the reduction in code that can be done using PyCUDA. The use of NumPy and general Python language can be used to write the non compute-intensive part of the program, while CUDA based kernels can perform the compute intensive code blocks. The low level CUDA APIs are exposed using the Driver and Device class in PyCUDA.

12.4.6 OpenCL (Open Compute Language)

OpenCL (Open Compute Language) is an open standard for parallel programming of heterogeneous systems. OpenCL standard is managed by the Khronos Group (same group who manages OpenGL standard). OpenCL is designed to run embedded systems, personal computers, and high-performance computing systems. OpenCL runs on CUDA for NVIDIA, and implementations of OpenCL for other GPUs are also available.

It is instructive to compare the compute kernel for vector addition (see Listing 12.4.4) with the OpenCL kernel shown below:

```
__kernel void AddVector( __global const float *A,
                         __global const float *B,
                         __global  float *SUM ) {
  int elementIndex = get_global_id(0);
```

```
5   SUM[elementIndex] = A[elementIndex] + B[elementIndex];
}
```

The key difference in the OpenCL kernel is the computation of thread index equivalent, elementIndex is done using function calls as opposed to implicit variables (as done in CUDA). OpenCL has functions to return the (i) work group index, (ii) work group size, (iii) thread index. The function used above `get_global_id` is used to get the id globally, while `get_local_id` is used to get the id within the work group. Starting with version 3.1 of the CUDA software, OpenCL is supported.

12.5 Conclusion

Parallel, especially multicore computing has become the mainstay of high-performance computing. More and more desktop computers also come equipped with multicore CPUs. In this chapter we have briefly introduced open-source tools and techniques to write efficient and scalable parallel programs. In this chapter we have discussed POSIX Threading library, OpenMP, MPI, Intel Thread Building Blocks, and GPU computing with CUDA and OpenCL. We expect parallel and multi-core computing to increase its impact on scientific as well as general computing in the years ahead.